

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра Систем информатики

Направление подготовки 09.06.01 – Информатика и вычислительная техника

ОТЧЕТ

Обучающегося Якутиной Светланы Александровны группы № 22215 курса 4
(Ф.И.О. полностью)

Тема задания: Разработка алгоритма для реализации метода Rag

Новосибирск 2025

Оглавление

Введение	3
Реализация алгоритма Retrieval-Augmented Generation.....	4
Листинг 1. Реализация основного RAG-конвейера	5
Листинг 2. Поиск релевантных элементов корпуса по косинусному схождению	6
Заключение.....	7

Введение

Целью данной лабораторной работы является разработка программного прототипа системы ответов на вопросы, использующей онтологию предметной области и подход Retrieval-Augmented Generation (RAG).

- Для достижения поставленной цели были выполнены следующие задачи:
- загрузка и анализ онтологии предметной области;
- преобразование элементов онтологического графа в текстовое представление;
- формирование корпуса и вычисление эмбедингов;
- реализация поиска релевантных узлов на основе косинусного сходства;
- интеграция языковой модели для генерации ответа на основе найденного контекста.

Реализация алгоритма Retrieval-Augmented Generation

Реализация алгоритма выполнена на языке Python. Общая схема работы системы включает несколько последовательных этапов.

На первом этапе загружается онтология из файла в формате JSON и формируются индексы графа, позволяющие эффективно получать исходящие и входящие связи для каждого узла. Для каждого объекта онтологии создаётся текстовое описание, включающее название, комментарий, атрибуты и часть связей с другими объектами.

На втором этапе из полученных текстов формируется корпус, для которого вычисляются векторные представления с использованием внешнего сервиса генерации эмбедингов. Эти представления используются для поиска наиболее релевантных фрагментов онтологии по заданному пользователем вопросу.

Для повышения качества поиска реализован механизм расширения запроса, при котором в исходный вопрос добавляются термины, наиболее близкие к нему по строковому сходству среди названий объектов онтологии.

Алгоритм Retrieval-Augmented Generation реализован в два шага. Сначала по расширенному вопросу выбираются наиболее близкие по эмбедингам элементы корпуса, на основе которых формируется промежуточный ответ языковой модели. Затем эмбединг этого ответа используется для дополнительного уточнения контекста и генерации финального ответа, опирающегося исключительно на факты, содержащиеся в онтологии.

Фрагмент кода, реализующий основной RAG-конвейер, приведён на листинге 1.

```

def rag_answer(question: str): 1 usage
    graph = load_graph(GRAPH_PATH)
    texts, nodes, all_labels = build_corpus(graph)

    corpus_embeddings = embed_texts(texts)

    # ---- ФАЗА 1 ----
    expanded_q = expand_question(question, all_labels, top_add=6)
    q_emb = np.asarray(EmbeddingUtils.get_embedding(expanded_q), dtype=np.float32)

    top_n_idx = cosine_topk(q_emb, corpus_embeddings, TOP_N)
    context_n = "\n\n".join(texts[int(i)] for i in top_n_idx)

    prompt_1 = (
        "Ответь на вопрос, используя ТОЛЬКО факты из текста.\n"
        "Если в тексте нет прямого факта – так и скажи.\n\n"
        f"Вопрос:\n{question}\n\n"
        f"Текст:\n{context_n}\n"
    )
    answer_1 = yandex_gpt(prompt_1)

    # ---- ФАЗА 2 ----
    a_emb = np.asarray(EmbeddingUtils.get_embedding(answer_1), dtype=np.float32)
    top_m_idx = cosine_topk(a_emb, corpus_embeddings, TOP_M)

    all_idx = list(dict.fromkeys([int(i) for i in top_n_idx] + [int(i) for i in top_m_idx]))
    context_nm = "\n\n".join(texts[i] for i in all_idx)

    prompt_2 = (
        "Дай финальный ответ на вопрос, используя ТОЛЬКО факты из текста.\n"
        "Старайся отвечать прямо: имя/объект + нужная связь.\n\n"
        f"Вопрос:\n{question}\n\n"
        f"Текст:\n{context_nm}\n"
    )
    final_answer = yandex_gpt(prompt_2)
    return final_answer

```

Листинг1. Реализация основного RAG-конвейера

Фрагмент кода реализующий поиск релевантных элементов корпуса представлен на листинге 2.

```
def cosine_topk(query_vec: np.ndarray, X: np.ndarray, k: int) -> np.ndarray:
    q = query_vec.astype(np.float32)
    X = X.astype(np.float32)

    q_norm = np.linalg.norm(q) + 1e-12
    X_norm = np.linalg.norm(X, axis=1) + 1e-12

    sims = (X @ q) / (X_norm * q_norm)
    top_idx = np.argsort(sims)[::-1][:k]
    return top_idx
```

Листинг 2. Поиск релевантных элементов корпуса по косинусному сходству

Примеры работы алгоритма:

Листинг3.

```
ВОПРОС: Кто признался в любви?
ОТВЕТ: Евгений Васильевич Базаров признался в любви.
```

Листинг4.

```
ВОПРОС: к какой эпохе относится произведение Отцы и дети
ОТВЕТ: Произведение «Отцы и дети» относится к эпохе 1860–1861 годов.
```

Листинг5.

```
ВОПРОС: к какой эпохе относится Война и мир
ОТВЕТ: Война и мир относится к эпохе войн против Наполеона в 1805–1812 годах.
```

Заключение

В ходе выполнения лабораторной работы был разработан программный прототип системы Retrieval-Augmented Generation, использующей онтологию предметной области в качестве источника знаний. Реализованы этапы преобразования онтологического графа в текстовый корпус, вычисления эмбедингов, поиска релевантных узлов и генерации ответа с помощью языковой модели.

Основную сложность представлял выбор формы текстового представления онтологических объектов и ограничение объёма контекста при сохранении информативности. Реализация двухэтапного поиска позволила повысить точность ответов и снизить влияние нерелевантных фрагментов.

Полученные результаты демонстрируют возможность использования онтологий в сочетании с векторными представлениями и языковыми моделями для построения систем вопросно-ответного типа.